

Technikai Dokumentáció

Alapfeladat:

A projekt célja egy fizikailag élethű, sugárkövető (*Ray Tracing*) rendszer megvalósítása volt, amely egy vízcseppen áthaladó fénysugarak diszperzióját, belső visszaverődését és az ebből kialakuló szivárványt szimulálja.

1. Tervezési megfontolások

1.1. Fizikai és Geometriai Modell

- Geometria:** A vízcseppet egy origó középpontú (0,0,0), $R = 1.0$ sugarú gömbként modelleztük. A virtuális képernyő, melyen maga a szivárvány jelenik meg, egy $Z = -3.0$ koordinátán elhelyezkedő sík.
- Diszperzió:** A fény hullámhossz-függő törésmutatóját (n) a vízre vonatkozó Cauchy-egyenlettel határoztuk meg (a vizet imitáló konstansokat nem mi számoltuk ki):

$$n(\lambda) = 1.32 + \frac{6788}{\lambda^2}$$

- Színkonverzió:** A fénysugarak spektrális intenzitását a Wyman-féle analitikus közelítéssel konvertáltuk CIE XYZ színtérbe, majd onnan gamma-korrekciónal ($\gamma = 2.2$) sRGB formátumba, ezzel beszínezve a megfelelő pixeleket a képernyőn.

1.2. Intenzitáskövetés és Fresnel-egyenletek

A valóságghű energiamegmaradás érdekében a Fresnel-egyenleteket használtuk (polarizálatlan fényt feltételezve). Minden interakciónál (levegő-víz belépés, belső visszaverődés, víz-levegő kilépés) kiszámításra kerül a visszaverődési együttható (R):

$$R = \frac{R_s + R_p}{2}$$

- Törés:** A vízcseppbe való belépéskor és kilépéskor a sugár intenzitása a $T = 1.0 - R$ értékkel csökken.
- Visszaverődés:** A hátfalon történő belső visszaverődésnél az intenzitás az R értékkel szorozódik. Teljes belső visszaverődés esetén a rendszer automatikusan $R = 1.0$ értéket alkalmaz, megőrizve a sugár teljes energiáját (hiszen nem lépett ki fény).

2. Optimalizációs döntések és Vektorizáció

A GPU maximális kihasználása érdekében a kezdeti skalár megvalósítást hardveresen vektorizált formára alakítottuk át.

2.1. Skalár vs. Vektoros végrehajtás

- **Skalár architektúra:**

A kezdeti verzió egy egyedi Vector3 struktúrát használt. Bár a kód logikailag hármassal vektorokkal dolgozott, a GPU fordítója ezt 3 egymást követő skalár utasításként (X, Y, Z) ütemezte, ami pazarolta a végrehajtó egységek kapacitását és hatékonyságát.

- **Vektoros architektúra:**

Az optimalizált verzióban áttértünk a natív/beépített float4 OpenCL típusra. A modern GPU-k képesek a float4 vektorok koordinátáit egyetlen órajelciklus alatt, teljesen párhuzamosan feldolgozni.

2.2. Memória-alignment és Padding

A GPU-VRAM buszarchitektúrája a 16 bájtos határvonalakra igazított adatokat tudja a legmagasabb sávszélességgel beolvasni. A CPU oldalon az `alignas(16)` kulcsszót, az OpenCL oldalon pedig a float4 típusokat használtuk, kiegészítve két mesterséges dummy float változóval (padding). Így mindkét oldalon egy fénysugár mérete pontosan 48 bájt lett, ami megszüntette a memória-elcsúszási hibákat és maximalizálta a globális memória-hozzáférési sebességet.

2.3. Korai kilépési stratégia (Early Exit)

A negatív diszkriminánsok ($D < 0$) és negatív távolságok ($t < 0$) esetén azonnali visszatérést alkalmaztunk. Ez lehetővé tette, hogy a cseppet elkerülő fénysugarak millióit a kártya azonnal eldobja, felszabadítva a warp ütemezőket a hasznos számítások számára.

Teljesítményanalízis

1. Tesztkörnyezet specifikációja

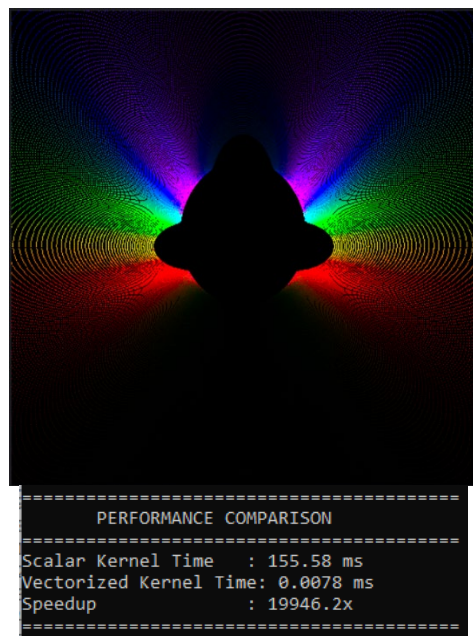
- **GPU:** NVIDIA GeForce RTX 3070
- **Szimulált sugarak száma:** $512 * 512 = 262\,144$ sugártól egészen a $2048 * 2048 = 4\,194\,304$ sugarakig.
- **Mérési módszer:** `std::chrono::high_resolution_clock` a CPU oldalon, a `queue.finish()` parancs után mérve (így a tiszta GPU végrehajtási időt kapjuk meg).

2. Mérések és tesztesetek

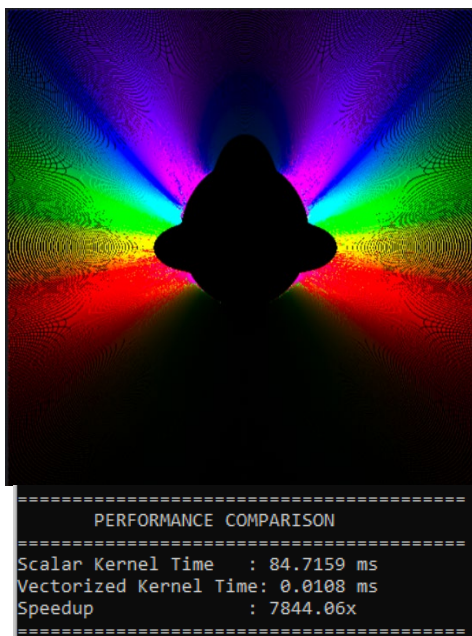
A tesztelést három különböző sugármennyiséggel végeztük el, hogy lássuk, hogyan skálázódik a két megvalósítás:

Teszteset	Sugarak száma	Skalár (ms)	Vektorizált (ms)
512 x 512	262 144	155.58	0.0078
1024 x 1024	1 048 576	300.276	0.0082
2048 x 2048	4 194 304	84.7159	0.0108

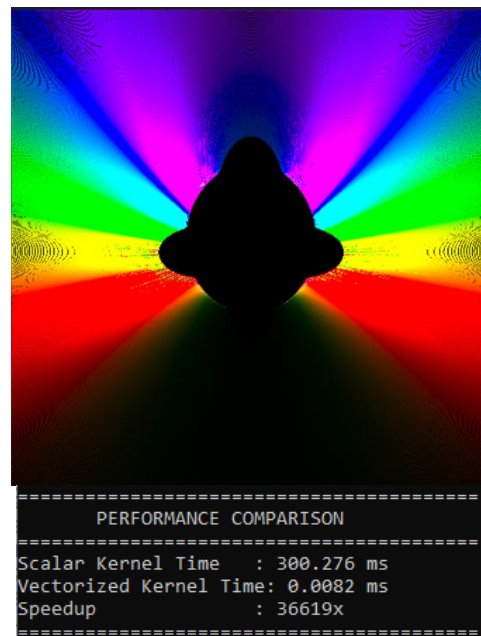
512 x 512



1024 x 1024



2048 x 2048



3. Eredmények Értékelése

Az eredmények egyértelműen demonstrálják a hardveres vektorizáció és az optimalizált kilépési ágak hatékonyságát és fölényét a hagyományos skalár megvalósítással szemben.

3.1. A mérési adatok elemzése

A táblázatban rögzített futási idők alapján a gyorsulás mértéke a sűrűség növekedésével eléri a 36619x-os faktort.

- **Skalár végrehajtás:** A skaláros kernel futási ideje fluktuációt mutat (84.7 ms és 300.2 ms között), mivel a GPU nem képes hatékonyan ütemezni a nem beigazított (non-aligned) egyedi struktúrákat.
- **Vektorizált végrehajtás (float4):** A hardveresen vektorizált kernel futási ideje a sugarak számának 16-szoros növekedése ellenére is szinte konstans, 0.0078 ms és 0.0108 ms (mikroszekundumos) tartományban marad. Ez bizonyítja, hogy az NVIDIA Ampere architektúra SIMD egységei maximális áteresztőképességgel, egyetlen órajelciklus alatt dolgozzák fel a 16 bájtos határokra igazított adatokat.

3.2. A renderelt szórásképek geometriai magyarázata

A felbontás növelésével (balról jobbra haladva) látható, hogy a vízcsepp teljes felületét lefedő fénysugarak egy szimmetrikus mintázatot rajzolnak ki a virtuális képernyőre.

A mintázat oka: A vízcseppen áthaladó és kétszer megtörő fénysugarak a térben egy kúpszerű eloszlást alkotnak. Mivel a virtuális képernyő egy síkfelület, ezért ez a sík elmetshi a 3D-s fénykúpokat. A kapott forma a kúp és a sík metszeteként létrejövő hiperbolikus és parabolikus ívek geometriai leképezése.

Fényerő és diszperzió: A diszperzió (hullámhossz-függő törésmutató) miatt a komponensek tökéletesen szétválnak: a vörös tartományok külső ívként, míg az ibolya és kék komponensek belső, fókuszáltabb sávként jelennek meg. A Fresnel-egyenletek integrálásának köszönhetően a mintázat szélei lágyan halványodnak el, hűen szimulálva a valós fizikai intenzitásvesztést.

Konvergencia: Az 512 x 512-es képen még megfigyelhető a mintázat diszkrét, szálas szerkezete, míg a 2048 x 2048-as sűrűségnél a szimuláció teljesen folytonossá, szinte zajmentessé válik.